

Node.js File Structure

- > config
- > controllers
- > middleware
- > models
- > node_modules
- > public
- > routes
- > uploads
- > utils
- > views

⚙ .env

{ } package-lock.json

{ } package.json

JS server.js

A well-organized file structure in a Node.js project is important for readability, scalability, and maintainability

Folder Roles Explained

- **controllers/** → Functions that define the logic for each route (e.g., `getUser`, `createPost`)
- **models/** → MongoDB or SQL schemas using `mongoose` or `sequelize`
- **routes/** → Define API endpoints and link them to controllers
- **middlewares/** → Code that runs between request and response (e.g., `JWT auth`, `logging`)
- **config/** → Database connection setup, environment config
- **utils/** → Utility/helper functions (e.g., `formatters`, `validators`)
- **public/** → Serve static files in Express
- **views/** → UI
- **.env** → Environment Variable like - `PORT` Number
- **server.js** → main root file it may be `app.js`, `index.js`

Best Practices

- Keep your `index.js` or `server.js` file clean – delegate logic to folders
- Group related files together (MVC structure is common)
- Always separate config and secret keys in `.env` file
-
- Use ES Modules or CommonJS consistently

